Singly Linked List – Delete At A Position2 –Space Complexity and Algorithm

Program:

```
void deleteAtPosition2(Node **head, int position)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    if (position < 0)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        Node *previous = nullptr;
        int i = 0;
        while (i < position && current != nullptr)
        {
            previous = current;
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
         {
             if (previous == nullptr)
             {
                 *head = current->next;
             }
             else
             {
                 previous->next = current->next;
             }
           cout << "Deleted node: " << current->data << endl;
           current->next = nullptr;
           free(current);
          current = nullptr;
           previous = nullptr;
           cout << "Node is deleted successfully." << endl;
         }
     }
}

…. .

   case 9:
         {
             cout << "Delete at a position,indexing node from 0 to N-1:" << endl;
             int position;
             cout << "Enter the node position to delete: ";
             cin >> position;
             deleteAtPosition2(&head, position);
             break;
         }
```

Space Complexity

Input Space:

A. Parameter – Only Definition:

- ***The input space consists of the parameters passed to the function: Node ** head and int position.***

- ***Both of these parameters require constant space, i.e., O(1), because they do not depend on the size of the linked list.***

  This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int position` (an integer), both of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

  • Node **head → one pointer (machine word)

  • int position → one integer (machine word) ⇒ Θ(1) (= O(1))

B. Full – Data (inclusive) Definition:

This more common definition includes the entire data structure. If the linked list has **n nodes** before the deletion, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same two machine words as above (Θ(1)) plus

• Every node that can be reached from *head at the moment the function is called. If the list currently contains n nodes, they occupy Θ(n) bytes. ⇒ Θ(n) (= O(n))

Auxiliary Space:

- ***Auxiliary space refers to the extra space or temporary space used by the function during its execution.***

- ***In this function, the auxiliary space is used for the temporary variables: Node * current, Node * previous, and int i.***

- $Each\ of\ these\ variables\ requires\ constant\ space, i.e., O(1)$.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. `Node *current`, `Node *previous`, and `int i`: The function declares two pointers and one integer to traverse the list and keep track of the position. Each of these variables occupies a constant amount of memory.
2. **Iteration**: The `while` loop reuses these three variables to find the correct node. No new memory is allocated during the traversal. This affects the function's **time complexity**, but not its space complexity.
3. `free(current)`: This function **releases memory** back to the system; it does not consume extra working space for the algorithm.

Since the function only needs a fixed number of extra variables (two pointers and an integer) to do its job, the amount of extra memory used is constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

• Local stack variables

– Node *current

– Node *previous

– int i

– a few compiler-generated temporaries

All together: constant size.

• No additional dynamic allocations (the single heap node being freed was part of the input).

• No recursion.

⇒ Θ(1) (= O(1))

$Algorithm$

$DELATPOS2(START, POS)$

1. $[LIST\ is\ Empty?]$

$If\ START = NULL, then:$

$Write:$ ``Error: List is empty.``

$Return\ and\ EXIT.$

$[End\ Of\ If\ Structure]$

2. $[POS < 0\ ?\ i.e.\ INDEX\ POSITION\ STARTS\ FROM\ 0]$

$if\ POS < 0\ , then:$

$Write:$ `` $Error: Invalid\ Position.$ ``

$Return\ and\ exit.$

$[Delete\ Node\ , When\ POS \geq 0\ ]$

$ELSE$

$SET\ CURRPTR := START\ \begin{bmatrix} CURRPTR\ is\ a\ pointer\ set\ to\ START, and\ will\ store \\ address\ of\ Deleted\ Node. \end{bmatrix}$

$SET\ PREVPTR := NULL\ \begin{bmatrix} PREVPTR\ is\ a\ pointer\ set\ to\ NULL\ and\ will\ store \\ address\ of\ PREVIOUS\ NODE\ of\ DELETED\ NODE. \end{bmatrix}$

$SET\ COUNT := 0[Why\ `0`\ as\ indexing\ starts\ from\ `0`\ here.]$

$\begin{bmatrix} THIS\ THE\ LOOP\ , When\ POSITION > 0\ , SET\ PREVPTR\ to\ PREVIOUS\ address\ of \\ DELETED\ NODE\ and\ CURRPTR\ to\ address\ of\ DELETED\ NODE. \end{bmatrix}$

$Repeat\ while\ COUNT < POS\ AND\ CURRPTR \neq NULL\ then:$

$SET\ PREVPTR := CURRPTR\ \begin{bmatrix} SET\ PREVPTR\ to\ the\ address\ , that\ CURRPTR \\ points\ to. \end{bmatrix}$

$SET\ CURRPTR := LINK[CURRPTR] \begin{bmatrix} SET\ CURRPTR\ to\ the\ address\ that \\ LINK\ of\ the\ NODE\ holds\ , and\ this \\ NODE\ to\ which\ CURRPTR\ is\ pointing. \end{bmatrix}$

$SET\ COUNT := COUNT + 1\ [INCREMENT\ COUNT]$

$[END\ OF\ LOOP]$

$$\left[\begin{array}{c}\textit{When POSITION is input by USER which is ``out of bounds.`` i.e. exceed to no. of nodes} \\ \textit{existed.}\end{array}\right]$$

$\textit{if CURRPTR} = \textit{NULL}, \textit{then}:$

$\textit{Write}: \text{``}\textit{Error}: \textit{INVALID POSITION}\text{``}$

$\textit{Return and exit}$

$[\textit{When POSITION input by USER which is not ``out of bounds.``}]$

$\textit{ELSE}$

$[\textit{When POSITION is } 0 \textit{ and PREVPTR}, \textit{then will remain at NULL}]$

$\textit{if PREVPTR} = \textit{NULL}, \textit{then}:$

$$\textit{SET START} := \textit{LINK}[\textit{CURRPTR}] \left[\begin{array}{c}\textit{SET START or HEAD to LINK of NODE that} \\ \textit{CURRPTR points to i.e. NULL, which} \\ \textit{delinks HEAD from that single NODE.}\end{array}\right]$$

$[\textit{When POSITION} > 0 \textit{ and PREVPTR} \neq \textit{NULL}]$

$\textit{ELSE}$

$$\textit{SET LINK}[\textit{PREVPTR}] := \textit{LINK}[\textit{CURRPTR}] \left[\begin{array}{c}\textit{Sets LINK of PREVIOUS NODE to} \\ \textit{address hold by LINK of NODE} \\ \textit{pointed by CURRPTR, which} \\ \textit{makes the PREVIOUS NODE} \\ \textit{points to that address.}\end{array}\right]$$

$[\textit{END of If Structure}]$

$$\textit{Write}: \text{"}\textit{Deleted node}: \text{"} \textit{ followed by INFO}[\textit{CURRPTR}] \left[\begin{array}{c}\textit{Print the data of the node} \\ \textit{being deleted.}\end{array}\right]$$

$\textit{SET LINK}[\textit{CURRPTR}] := \textit{NULL}\ [\textit{SET LINK of NODE pointed by CURRPTR to NULL}]$

$\textit{FREE}(\textit{CURRPTR})[\textit{Deallocate the memory used by the node.}]$

$\textit{SET CURRPTR} := \textit{NULL}\ [\textit{Sets Dangling pointer CURRPTR to NULL}]$

$\textit{SET PREVPTR} := \textit{NULL}\ [\textit{Sets Dangling pointer PREVPTR to NULL}]$

$\textit{Write}: \text{"}\textit{Node is deleted successfully.}\text{"}\ [\textit{Write Statement on Confirmation of Deletion}]$

$[\textit{END OF IF STRUCTURE}]$